AI-ASSISTED DEVELOPMENT / DEVELOPER EDUCATION

Technical Content Creator Portfolio & Writing Sample

PROJECT CASE STUDIES + PRACTICAL CODEX TUTORIAL

Prepared for technical content, developer education and AI enablement applications.

POSITIONING

I work at the intersection of AI-assisted software development, product delivery and clear technical communication. My project experience spans a cultural commerce marketplace, an AI content platform and client website delivery. I translate requirements into structured tasks, work with AI coding assistants, review changes and coordinate testing and release preparation.

What this portfolio demonstrates

- Explaining technical tools and workflows to beginner and intermediate audiences.
- Documenting real projects with clear distinctions between completed work, ongoing work and proposed features.
- Applying structured prompts, reviews, tests and approval steps in AI-assisted development.
- Communicating product decisions across user experience, infrastructure and delivery.

Project overview

KULTRAE	Cultural fashion marketplace. GitHub, Codex, Supabase, staging and payment-flow planning.
Sevorify	AI-driven social content platform. Product requirements, dashboard UX, workflow planning and repository transition.
Website projects	Client website requirements, responsive interface improvements and practical troubleshooting.
Writing sample	A beginner-friendly Codex workflow for investigating and safely fixing a mobile navigation issue.

Evidence note: This is a curated narrative based on described project work, not an independently audited deployment record. Before submitting, add screenshots, links and any supporting pull requests or test reports.

01 KULTRAE | Marketplace development

CASE STUDY • ONGOING PRODUCT

KULTRAE is a UK-focused cultural fashion marketplace. The project covers discovery, user experience, listing and seller workflows, marketplace infrastructure, checkout design and staged release preparation.

Problem and product goal

Deliver a reliable marketplace experience for shoppers and sellers while keeping payment, seller identity and launch configuration risks controlled.

My role and contribution

- Defined product requirements, reviewed design concepts and prioritised changes across listing, checkout, seller and admin screens.
- Directed AI-assisted engineering tasks through Codex and reviewed reported results, issues and next actions.
- Coordinated a separate staging approach, with a visible test banner, noindex configuration and distinct test services.
- Tracked readiness through GitHub pull requests, deployment controls and release checks.

Technical evidence available to include

- Supabase marketplace database migrations and row-level security configuration.
- Staging preparation and UK/GBP checkout rules.
- A GitHub fix for buyer navigation to seller profiles, plus release-branch review and CI history.

What this demonstrates

The ability to write actionable engineering briefs, communicate technical risks, separate testing from production and explain why each verification step matters.

Suggested supporting exhibits

A. A redacted GitHub pull request and successful CI screenshot. B. Staging banner screenshot. C. Annotated seller-profile bug before/after. D. A one-page requirement-to-test trace.

Project repository: https://github.com/kultrae-fashion/kultrae-marketplace (access and visibility should be checked before sharing).

02 Sevorify | AI content platform

CASE STUDY • PRODUCT DESIGN AND DEVELOPMENT

Sevorify is an AI-assisted content engine concept designed to help businesses create and adapt social posts from their photos and videos, with editing, scheduling and publication workflows.

Problem and product goal

Business users need a clear path from uploading original media to producing, approving and scheduling platform-ready content. The interface must make AI-generated output understandable and editable.

My role and contribution

- Defined an end-to-end content workflow from media intake through creation, review, scheduling and publishing.
- Reviewed dashboard and information architecture concepts and selected a midnight/navy interface with violet and pink accents.
- Identified mobile navigation usability problems and requested responsive design improvements.
- Planned a safer transition from an AI-built codebase into a GitHub/Codex workflow for ongoing maintenance.

Documentation opportunities

- Getting started: how a client uploads media and creates the first content draft.
- Feature explainer: generated draft, human edits, approval status and scheduling state.
- Troubleshooting: diagnosing a navigation link that does not respond on mobile.

What this demonstrates

Product thinking, audience-focused explanations, workflow decomposition and a practical understanding of human review in AI-supported content systems.

Project repository: https://github.com/Severify/severify-platform (verify access and public sharing rights).

03 Website projects | Client-facing delivery

CASE STUDY • SELECTED WEBSITE WORK

Client website work has involved capturing requirements, reviewing page designs, preparing responsive interfaces, communicating revisions and addressing mobile usability issues.

Representative workflow

1. Clarify scope: intended audience, pages, content, integrations and exclusions.
2. Build and review: inspect desktop and mobile layouts against agreed requirements.
3. Test and repair: reproduce issues, isolate likely causes, verify links and retest affected screens.
4. Document handover: state what changed, what was tested and what remains with the client.

Evidence to attach

Add one named client project with permission, before-and-after screenshots, the delivered URL if public, and a short handover or QA checklist. Do not claim launch or production ownership without supporting records.

04 Technical writing sample

TUTORIAL • BEGINNER • 20–30 MINUTE GUIDED EXERCISE

How to use Codex to investigate and fix a mobile navigation issue

Audience: beginner website builders using GitHub and an AI coding assistant. Learning goal: understand a safe workflow for asking Codex to inspect a repository, propose a focused fix, verify results and explain the change.

What you need

- A website repository that you are authorised to edit, opened in a Codex-supported environment.
- A separate working branch, plus the project's normal install and test instructions.
- A reproducible issue, such as a menu link failing to respond on a phone-sized screen.

Step 1 | Define the symptom

Test the problem yourself. Note the page, approximate viewport width, exact interaction and expected result. Example: "At a 390 px viewport, tapping the navigation menu opens the panel, but the Services link does not navigate." This is a fictional teaching example, not a reported Sevorify test result.

Step 2 | Ask Codex to investigate without editing

Copy this prompt into Codex:

PROMPT Inspect the mobile navigation implementation in this repository. Do not edit files yet. Identify likely causes of a visible but unclickable menu link. Report relevant files, your evidence, the smallest proposed fix and the checks needed to validate it. Do not change deployment settings or secrets.

Why this helps: requesting evidence before edits reduces the risk of a speculative fix and gives the developer an opportunity to approve the plan.

Step 3 | Review the proposed change

Read the file references and proposed cause. Confirm the fix targets the mobile menu rather than unrelated layout or styling. If the evidence is unclear, ask Codex for a reproduction or additional inspection.

Step 4 | Authorise a small implementation

PROMPT Apply only the approved mobile navigation fix. Follow existing project conventions. Add or update a regression test where feasible. Run the project's applicable lint, typecheck, tests and build. Show changed files, test commands and outcomes. Do not commit, push or deploy.

Tutorial | Validation and teaching notes

Step 5 | Check the evidence

Review the diff. Check whether Codex edited only the files needed for the fix. Review the real output of applicable commands, rather than relying on a plain-language “fixed” message. If a command was unavailable or failed, record the limitation.

Step 6 | Test in the browser

Repeat the exact failing interaction at a phone-sized viewport. Also check desktop navigation, keyboard focus and related pages. A passing build alone does not prove that a tap interaction works.

Step 7 | Document the result

Write a short release note using the following fields: issue, root cause supported by evidence, files changed, validation performed, remaining risks and deployment status. Only mark the bug resolved after independently reproducing a successful interaction.

Worked example: completion note template

Issue	On a narrow screen, a navigation link was unresponsive.
Investigation	[Insert observed DOM / CSS / event-handler evidence.]
Change	[Insert exact files and smallest correction.]
Automated checks	[List command, result and date for each check.]
Manual test	[Record device / viewport, clicks and observed behaviour.]
Status	[Verified locally / awaiting QA / deployed, as applicable.]

Common mistakes and prevention

- Mistake: asking for a full redesign to fix one link. Better: require the smallest scoped change.
- Mistake: accepting a success summary without reading the diff. Better: inspect exact files and test output.
- Mistake: testing only desktop. Better: reproduce the original mobile failure and recheck accessibility.
- Mistake: deploying before approval. Better: keep commits and deployment as separate explicit decisions.

Knowledge check

5. Why should the first Codex prompt be read-only?
6. Why does a passing build not prove that a mobile link works?
7. Which evidence belongs in a completion note?

Suggested answers: investigation reduces unnecessary edits; a build checks compilation, not every browser interaction; evidence includes files changed, actual test outcomes, browser checks and deployment state.

Further reading

OpenAI developer guidance: https://developers.openai.com/blog/run-long-horizon-tasks-with-codex

OpenAI prompting reference: https://developers.openai.com/api/docs/guides/prompt-engineering

05 Submission checklist

EVIDENCE TO ADD BEFORE SHARING

- Your preferred professional name, contact email and LinkedIn URL on the cover.
- 2–3 original screenshots from KULTRAE, with brief annotations.
- 2 original screenshots from Sevorify, clearly marked prototype or running build.
- One named website case study, permitted client name or anonymised sector, live URL if available.
- A public-safe pull request, commit link or sanitised diff showing a reviewed change.
- A completed version of the tutorial with real screenshots, commands and outputs from a controlled demo repository.
- Check all project claims and repo permissions before sending to an employer.

Suggested accompanying application statement

My experience combines AI-assisted software development, technical project delivery and a strong interest in developer education. Through KULTRAE, Sevorify and website projects, I have defined requirements, worked with AI coding tools, coordinated GitHub workflows and reviewed testing and release activities. I enjoy making technical processes understandable. The included Codex tutorial demonstrates how I structure guidance around learning outcomes, safe implementation and evidence-based validation.

Portfolio edition: November 2026. Narrative draft and instructional example prepared for review; placeholders require evidence before final submission.